CAPSTONE PROJECT REPORT CONTENT

Online Fraud Transaction Detection System Using Machine Learning, FastAPI and MLOps

1. Abstract

Financial fraud has become one of the most significant challenges faced by financial institutions and digital payment service providers. With the rapid growth of online transactions, it is essential to develop automated systems capable of distinguishing fraudulent activities from legitimate transactions in real time. The objective of this project was to design, develop, evaluate, and deploy an end-to-end machine learning solution for detecting fraudulent financial transactions.

The project utilizes the online fraud transaction dataset containing over 6.3 million transaction records. Comprehensive exploratory data analysis, data cleaning, feature engineering, and preprocessing techniques were applied to prepare the dataset for modeling. Three machine learning algorithms, namely Logistic Regression, Decision Tree, and Random Forest, were developed and evaluated. The Random Forest Classifier demonstrated superior performance and was selected as the final model.

The trained model was serialized using Joblib and deployed through FastAPI, enabling real-time fraud prediction through REST APIs. Additionally, GitHub and GitHub Actions were utilized to implement version control and support MLOps practices. The project successfully demonstrates the complete machine learning lifecycle, from data preparation to deployment, while addressing class imbalance, large-scale data processing, and real-world deployment requirements.

2. Introduction

The digital banking and financial technology sectors process millions of transactions daily. While online financial services offer convenience and efficiency, they also increase the risk of fraudulent transactions. Fraudulent transactions can lead to considerable financial losses, reputational damage, and regulatory consequences for organizations.

Traditional rule-based fraud detection systems often struggle to identify complex fraud patterns and adapt to evolving attack strategies. Machine learning techniques provide a powerful alternative by analyzing historical transaction behavior and detecting anomalies that indicate potential fraud.

This project focuses on developing a machine learning-based fraud detection system capable of identifying fraudulent transactions with high accuracy and recall. The system is designed to support real-time predictions through API deployment and follows modern MLOps practices for maintainability and scalability.

3. Problem Statement

The primary objective of this project is to build an intelligent fraud detection system capable of classifying financial transactions as either:

- Legitimate Transaction (0)
- Fraudulent Transaction (1)

The system should:

- Detect fraudulent transactions accurately.
- Minimize false negatives.
- Handle highly imbalanced data.
- Support real-time prediction.
- Be deployable through APIs.

4. Business Objective

The key business objectives of the project are:

- Reduce financial losses caused by fraudulent transactions.
- Improve fraud detection efficiency.
- Automate transaction monitoring.
- Support decision-making through predictive analytics.
- Provide a scalable API-based fraud detection solution.

5. Dataset Description

```
[4]  fraud_df.info()

...  <class 'pandas.DataFrame'>
     RangeIndex: 6362620 entries, 0 to 6362619
     Data columns (total 11 columns):
      #   Column          Dtype
     ---  ------          -----
      0   step            int64
      1   type            str
      2   amount          float64
      3   nameOrig        str
      4   oldbalanceOrg   float64
      5   newbalanceOrig  float64
      6   nameDest        str
      7   oldbalanceDest  float64
      8   newbalanceDest  float64
      9   isFraud         int64
      10  isFlaggedFraud  int64
     dtypes: float64(5), int64(3), str(3)
     memory usage: 706.2 MB
```

The project uses the Fraud Detection Dataset.

Dataset Statistics

Attribute	Value
Total Records	6,362,620
Original Features	11
Target Variable	isFraud
Fraud Transactions	8,213
Non-Fraud Transactions	6,354,407
Fraud Percentage	0.129%

Original Features

Feature Name	Description
step	Unit of time
type	Transaction type
amount	Transaction amount
nameOrig	Sender account
oldbalanceOrg	Initial balance of sender
newbalanceOrig	New balance of sender
nameDest	Receiver account
oldbalanceDest	Initial balance of receiver
newbalanceDest	New balance of receiver
isFraud	Fraud indicator
isFlaggedFraud	Flagged fraud indicator

6. Exploratory Data Analysis (EDA)

After loading the dataset, extensive exploratory data analysis was performed.

Dataset Structure

Rows : 6,362,620
Columns : 11
Memory Usage : 706.2 MB

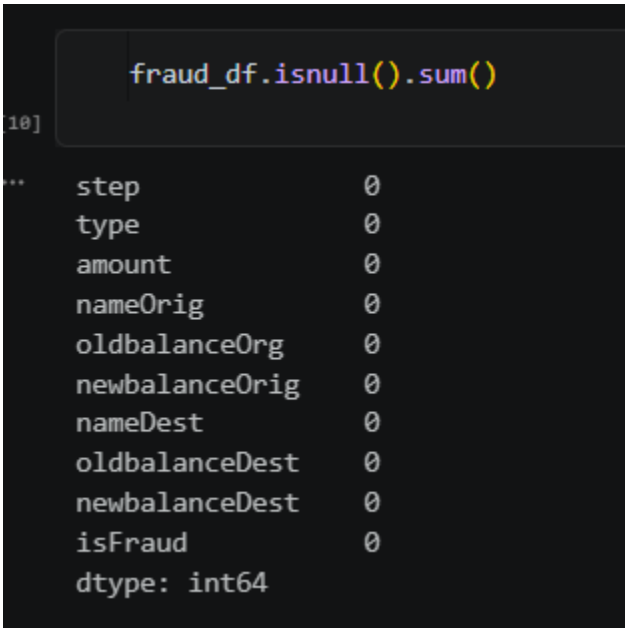
Missing Values Analysis

The dataset was checked for missing values.

Finding

Plain Text
Missing Values = 0

No missing values were present in any feature.



Duplicate Records Analysis

Duplicate transaction records were analyzed.

Finding

Plain Text
Duplicate Records = 0

No duplicate records were found.

```
[1]  fraud_df.duplicated().sum()

     np.int64(0)
```

Class Distribution Analysis

The dataset was found to be highly imbalanced.

Plain Text

Non-Fraud Transactions = 6,354,407

Fraud Transactions = 8,213

Fraud percentage:

Plain Text

0.129%

```
[7]  fraud_df['isFraud'].value_counts()

     isFraud
     0    6354407
     1       8213
     Name: count, dtype: int64
```

Transaction Type Analysis

The distribution of transaction types is shown below.

Transaction Type	Count
CASH_OUT	2,237,500
PAYMENT	2,151,495
CASH_IN	1,399,284
TRANSFER	532,909
DEBIT	41,432

Observations:

- CASH_OUT transactions were most frequent.
- PAYMENT transactions accounted for a significant proportion.
- DEBIT transactions were relatively rare.
- Fraudulent behavior was more prominent in specific transaction categories.

7. Data Cleaning and Feature Selection

Removal of isFlaggedFraud

The feature `isFlaggedFraud` contained only 16 positive records.

Plain Text
0 = 6,362,604
1 = 16

Due to extremely low variance and limited predictive contribution, the feature was removed.

Python
fraud_df.drop(columns=['isFlaggedFraud'])

Removal of nameOrig and nameDest

The features:

Plain Text
nameOrig
nameDest

represent account identifiers.

These columns were removed because:

- Extremely high cardinality.
- Mostly unique values.
- Increased dimensionality.
- Limited predictive value.

Python

fraud_df.drop(columns=['nameOrig','nameDest'])

8. Feature Engineering

To improve fraud detection performance, four new features were engineered.

```
fraud_df["orig_balance_diff"] = (
    fraud_df["oldbalanceOrg"]
    - fraud_df["newbalanceOrig"]
)

fraud_df["dest_balance_diff"] = (
    fraud_df["newbalanceDest"]
    - fraud_df["oldbalanceDest"]
)

fraud_df["orig_error"] = (
    fraud_df["oldbalanceOrg"]
    - fraud_df["amount"]
    - fraud_df["newbalanceOrig"]
)

fraud_df["dest_error"] = (
    fraud_df["oldbalanceDest"]
    + fraud_df["amount"]
    - fraud_df["newbalanceDest"]
)

print("Feature Engineering Completed!")
```

8.1 Origin Balance Difference

Python
orig_balance_diff =
oldbalanceOrg - newbalanceOrig

This feature measures the balance reduction in the sender account after a transaction.

8.2 Destination Balance Difference

Python
dest_balance_diff =
newbalanceDest - oldbalanceDest

This feature measures the balance increase in the receiver account.

8.3 Origin Error

Python
orig_error =oldbalanceOrg - amount - newbalanceOrig

This feature identifies inconsistencies in sender account balance calculations.

8.4 Destination Error

Python
dest_error = oldbalanceDest + amount - newbalanceDest

This feature captures anomalies in receiver account balance updates.

Final Feature Set

The final model used the following eleven features:

Plain Text
step
type
amount
oldbalanceOrg
newbalanceOrig

oldbalanceDest
newbalanceDest
orig_balance_diff
dest_balance_diff
orig_error
dest_error

9. Handling Class Imbalance

Fraud detection datasets are naturally imbalanced.

Training Distribution

Plain Text
Class 0 = 99.87%
Class 1 = 0.13%

Instead of oversampling techniques such as SMOTE, the project utilized:

Python
class_weight='balanced'

Advantages:

- Preserves original data distribution.
- Avoids synthetic observations.
- Reduces computational cost.
- Works effectively with Random Forest.

10. Data Splitting Strategy

The dataset was divided using stratified sampling.

Python
train_test_split(
test_size=0.3,

random_state=10,
stratify=y
)

Train-Test Ratio

Plain Text
Training : 70%
Testing : 30%

Stratification ensured the class distribution remained consistent across both datasets.

11. Training Sample Optimization

Due to the large dataset size, a stratified training sample was generated.

Python
train_size = 500000
``

Sample Size

Plain Text
X_train_sample : 500,000 rows
y_train_sample : 500,000 rows

Benefits:

- Faster training.
- Reduced memory consumption.
- Preserved class distribution.

12. Data Preprocessing Pipeline

A machine learning pipeline was created to automate preprocessing.

```
num_pipeline = Pipeline(
    steps=[
        ("scaler", StandardScaler())
    ]
)

cat_pipeline = Pipeline(
    steps=[
        (
            "onehot",
            OneHotEncoder(handle_unknown="ignore")
        )
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", num_pipeline, num_cols),
        ("cat", cat_pipeline, cat_cols)
    ]
)

print("Preprocessing Pipeline Created!")
```

Numerical Pipeline

Python
StandardScaler()

Purpose:

- Normalize feature scales.
- Improve numerical stability.

Categorical Pipeline

Python
OneHotEncoder(handle_unknown='ignore')

Purpose:

- Convert transaction types into numerical vectors.

Column Transformer

Python
ColumnTransformer()

Combined categorical and numerical transformations.

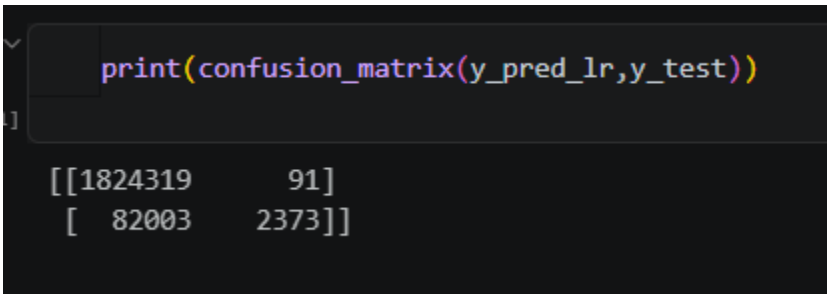
13. Machine Learning Models

Three classification algorithms were implemented.

13.1 Logistic Regression

Python
LogisticRegression(
class_weight="balanced"
)

Results



```
print(classification_report(y_pred_lr,y_test))
```

```
              precision    recall  f1-score   support

           0       0.96      1.00      0.98   1824410
           1       0.96      0.03      0.05     84376

    accuracy                           0.96   1908786
   macro avg       0.96      0.51      0.52   1908786
weighted avg       0.96      0.96      0.94   1908786
```

Accuracy : 96%

Recall : 3%

F1 Score : 0.05

The model struggled to identify fraudulent transactions.

13.2 Decision Tree Classifier

Python

DecisionTreeClassifier(

class_weight='balanced'

)

Results

```
print(confusion_matrix(y_pred_dt,y_test))
```

```
[[1906309      14]
 [     13    2450]]
```

```
print(classification_report(y_pred_dt,y_test))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00   1906323
           1       0.99      0.99      0.99      2463

    accuracy                           1.00   1908786
   macro avg       1.00      1.00      1.00   1908786
weighted avg       1.00      1.00      1.00   1908786
```

Precision : 0.99

Recall : 0.99

F1 Score : 0.99

Performance improved significantly over Logistic Regression.

13.3 Random Forest Classifier

Python

```
RandomForestClassifier(
n_estimators=100,
max_depth=15,
class_weight='balanced',
random_state=10,
n_jobs=-1
)
```

Results

```
print(confusion_matrix(y_pred_rf,y_test))
```

```
[[1906320      38]
 [      2    2426]]
```

```
print(classification_report(y_pred_rf,y_test))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00   1906358
           1       0.98      1.00      0.99      2428

    accuracy                           1.00   1908786
   macro avg       0.99      1.00      1.00   1908786
weighted avg       1.00      1.00      1.00   1908786
```

Precision : 0.98

Recall : 1.00

F1 Score : 0.99

The Random Forest model achieved the best overall performance.

14. Model Evaluation

Random Forest Confusion Matrix

Plain Text

[[1906320 38]

[2 2426]]

Interpretation

- True Negatives = 1,906,320
- False Positives = 38
- False Negatives = 2
- True Positives = 2,426

The model successfully identified almost every fraudulent transaction.

Classification Report

Fraud Class (Class 1)

Metric	Value
Precision	0.98
Recall	1.00
F1 Score	0.99

Overall Accuracy

Plain Text
100%

15. Final Model Selection

The Random Forest model was selected because:

- Highest Recall
- Highest F1 Score
- Excellent Precision
- Robust to nonlinear relationships
- Effective on imbalanced data
- Reduced false negatives

16. Model Serialization

The final model pipeline was saved using Joblib.

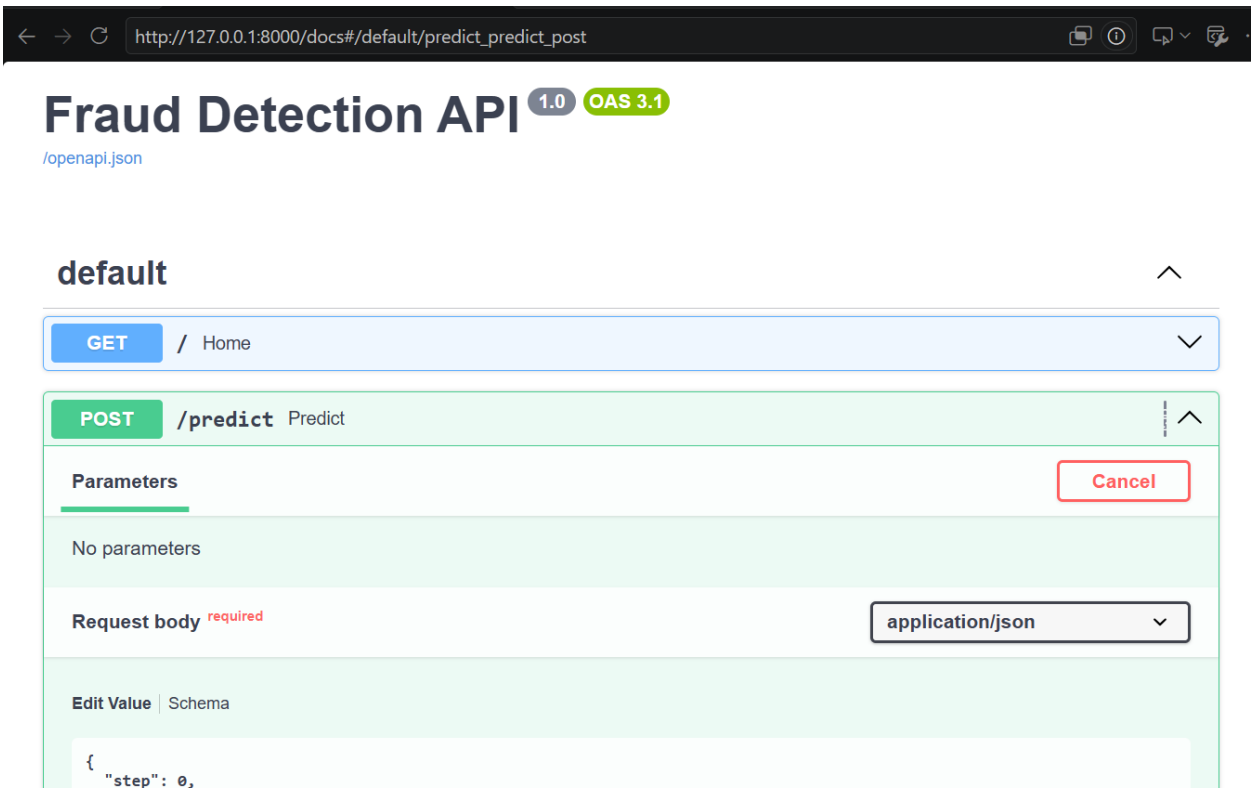
Python
joblib.dump(
rf_pipeline,
"./model/fraud_detector.pkl"
)

Advantages:

- Easy deployment.
- Reproducibility.
- Consistent inference.

17. FastAPI Deployment

A FastAPI application was developed for real-time fraud prediction.



Endpoints

Home Endpoint

HTTP
GET /

Response:

JSON

```
{
"message":"Fraud Detection API"
}
```

Prediction Endpoint

HTTP

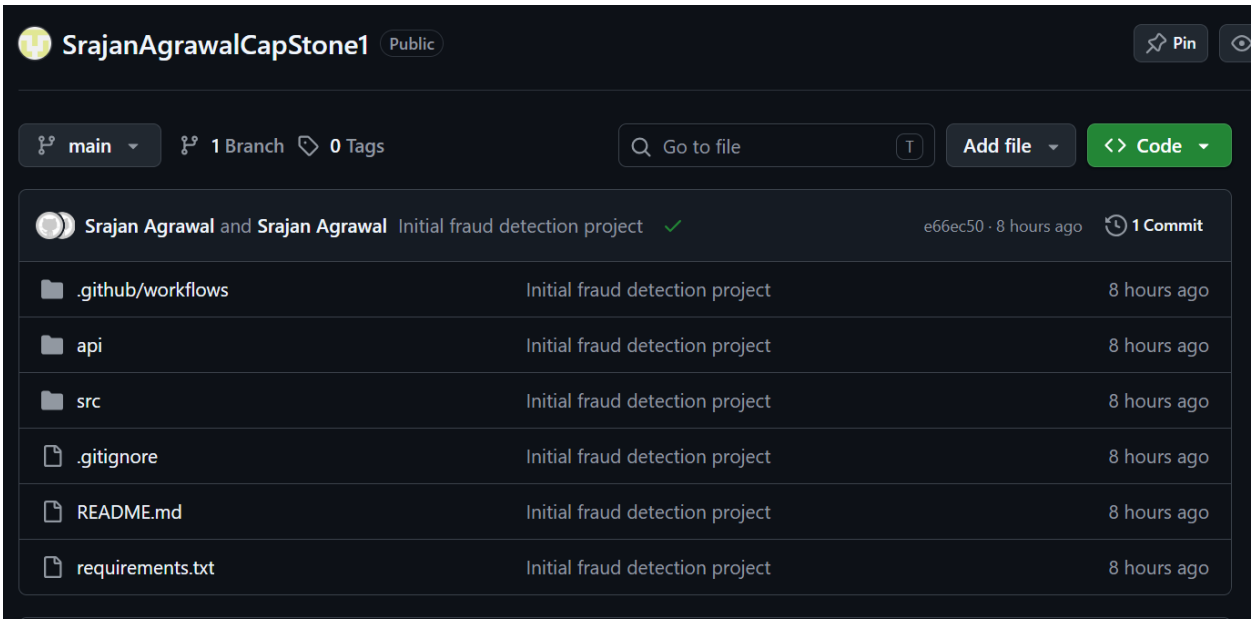
```
POST /predict
```

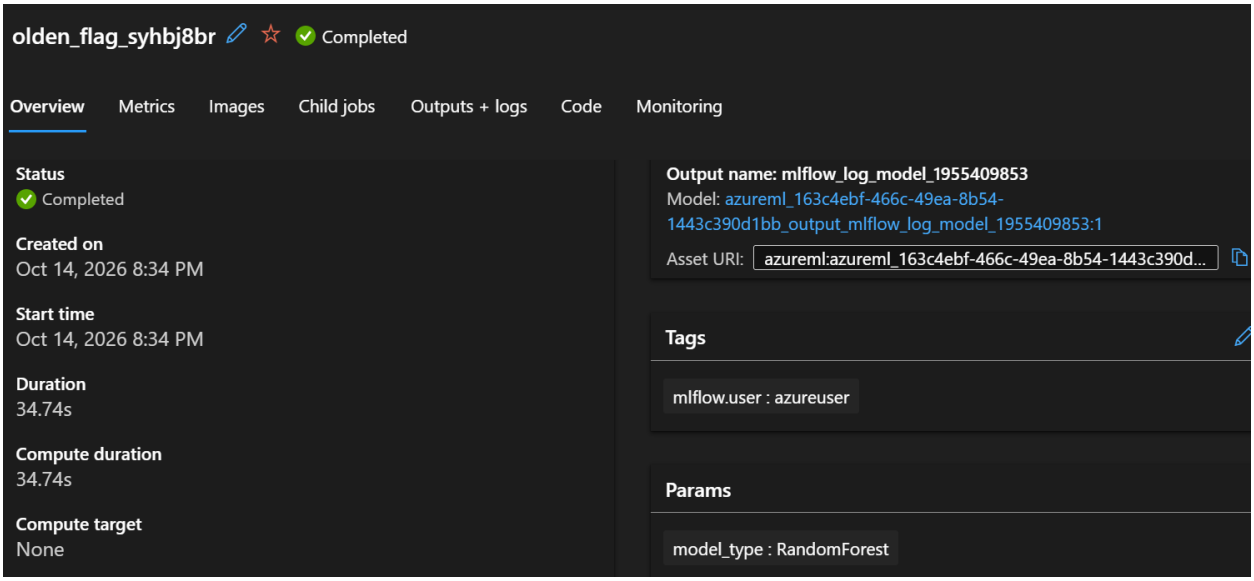
The API accepts transaction information and returns:

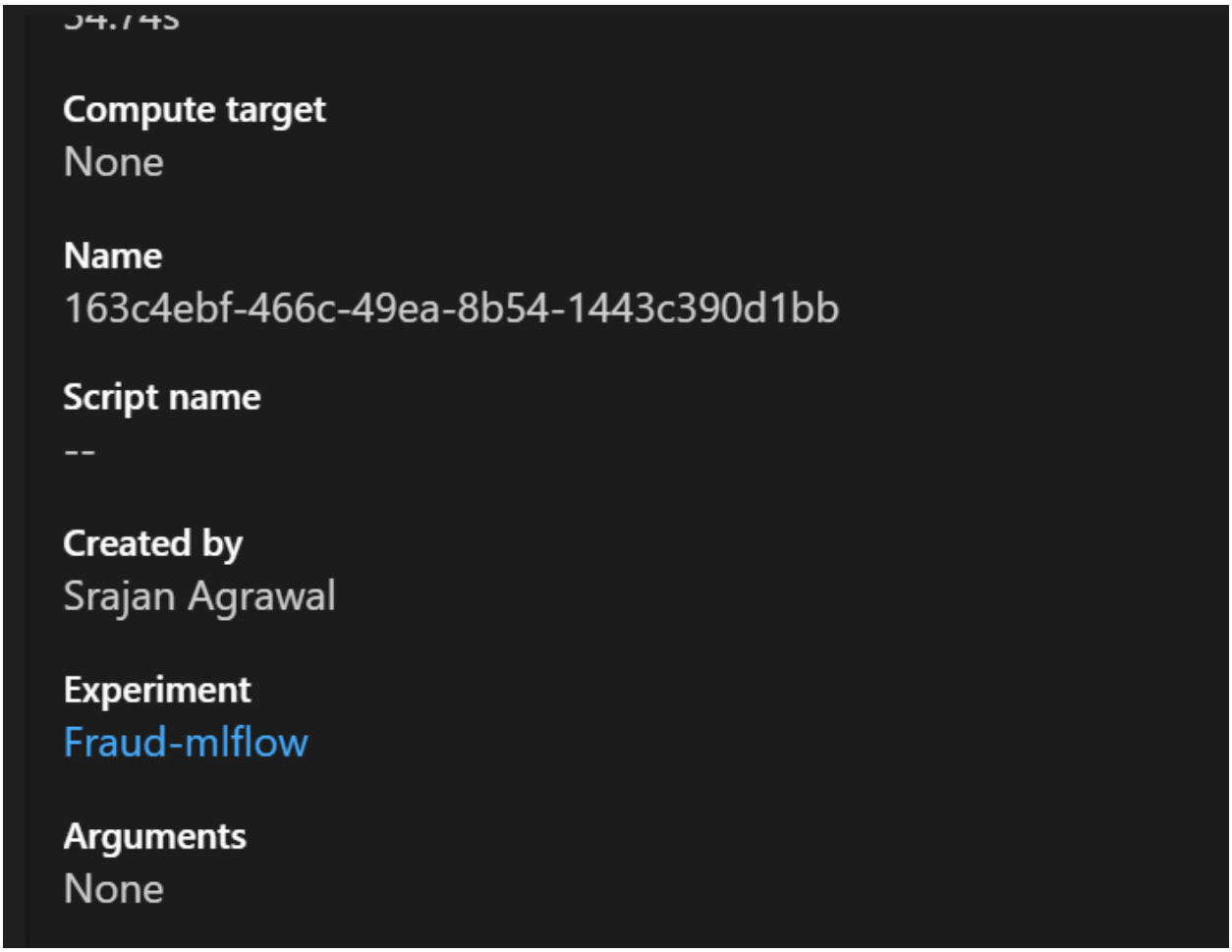
JSON

```
{
"prediction": 1,
"fraud_probability": 0.98
}
```

18. MLOps Implementation


SrajanAgrawalCapStone1
Public
Pin
main
1 Branch
0 Tags
Go to file
T
Add file
Code
Srajan Agrawal and Srajan Agrawal Initial fraud detection project
e66ec50 · 8 hours ago
1 Commit
.github/workflows
Initial fraud detection project
8 hours ago
api
Initial fraud detection project
8 hours ago
src
Initial fraud detection project
8 hours ago
.gitignore
Initial fraud detection project
8 hours ago
README.md
Initial fraud detection project
8 hours ago
requirements.txt
Initial fraud detection project
8 hours ago


olden_flag_syhbj8br
Completed
Overview
Metrics
Images
Child jobs
Outputs + logs
Code
Monitoring
Status
Completed
Created on
Oct 14, 2026 8:34 PM
Start time
Oct 14, 2026 8:34 PM
Duration
34.74s
Compute duration
34.74s
Compute target
None
Output name: mlflow_log_model_1955409853
Model: azureml_163c4ebf-466c-49ea-8b54-1443c390d1bb_output_mlflow_log_model_1955409853:1
Asset URI: azureml:azureml_163c4ebf-466c-49ea-8b54-1443c390d...
Tags
mlflow.user : azureuser
Params
model_type : RandomForest



The project incorporates MLOps concepts through:

Version Control

- Git
- GitHub

CI/CD

- GitHub Actions

Model Management

- Joblib Serialization

Deployment

- FastAPI

Modular Project Design

Plain Text

```
src/
→ train.py
→ prediction.py
→ evaluate.py
```

19. Project Architecture

Plain Text

```
Data Collection
↓
Data Cleaning
↓
Feature Engineering
↓
Train-Test Split
↓
Preprocessing Pipeline
↓
Model Development
↓
Model Evaluation
↓
Model Serialization
↓
FastAPI Deployment
↓
Real-Time Prediction
```

20. Challenges Faced

- Severe class imbalance.
- Large dataset size.
- Computational resource limitations.
- High-cardinality categorical variables.
- Repository size limitations on GitHub.
- Deployment pipeline integration.

21. Future Enhancements

- Azure MLflow integration.
- Streamlit dashboard.
- Docker containerization.
- Explainable AI using SHAP.
- Real-time monitoring.
- Automated retraining pipelines.
- Azure cloud deployment.

22. Conclusion

This project successfully developed a complete end-to-end fraud detection solution using machine learning and MLOps practices. Through extensive data analysis, feature engineering, preprocessing, model development, evaluation, and deployment, a highly effective fraud detection model was achieved. The Random Forest Classifier demonstrated exceptional performance with near-perfect fraud detection capability and was successfully deployed through FastAPI. The project provides a scalable, production-oriented framework for financial fraud detection and lays the foundation for future cloud-native machine learning solutions.